

RAPPORT DE PROJET : Système de Surveillance IoT avec Cloud Firebase

Sujet : Automatisation, Communication Série (UART) et Cloud Computing

Technologie : Arduino Uno, ESP32, Firebase (Google)

Réalisée par : Jaafar Marfouq/Amine Mokid/Rayane Mahtaj

1. Introduction

1.1 Contexte et Motivation

Dans l'industrie 4.0, la remontée d'informations vers le Cloud est essentielle pour l'analyse de données à long terme. Cependant, de nombreux systèmes industriels reposent sur des automates robustes (comme l'Arduino) qui gèrent les capteurs mais manquent de connectivité native. Ce projet vise à créer un pont entre le monde physique (Arduino) et le monde numérique (Cloud Firebase) pour une surveillance thermique efficace.

1.2 Objectif du Projet

Ce projet a pour but de réaliser un système complet capable de :

1. **Gérer Localement (Edge Computing) :** L'Arduino mesure la température, affiche les données sur un LCD, et active le refroidissement (ventilateur) et l'alarme (buzzer) si nécessaire.
2. **Transmettre :** L'Arduino envoie les données de température en temps réel via une liaison série (UART).
3. **Connecter (Gateway) :** L'ESP32 reçoit ces données et les envoie vers une base de données en temps réel **Google Firebase** pour un suivi à distance.

1.3 Architecture du Système

Le système est divisé en deux blocs fonctionnels distincts :

- **Le Bloc de Contrôle (Arduino Uno) :** Il est autonome. Même si le Wi-Fi coupe, l'Arduino continue de refroidir le système et d'alerter localement. Il gère toute la logique "métier".
- **Le Bloc de Communication (ESP32) :** Il agit comme une passerelle transparente. Il écoute l'Arduino et répète tout ce qu'il entend vers Firebase.

2. Liste du Matériel et Rôles

Les composants sont désormais centrés autour de l'Arduino.

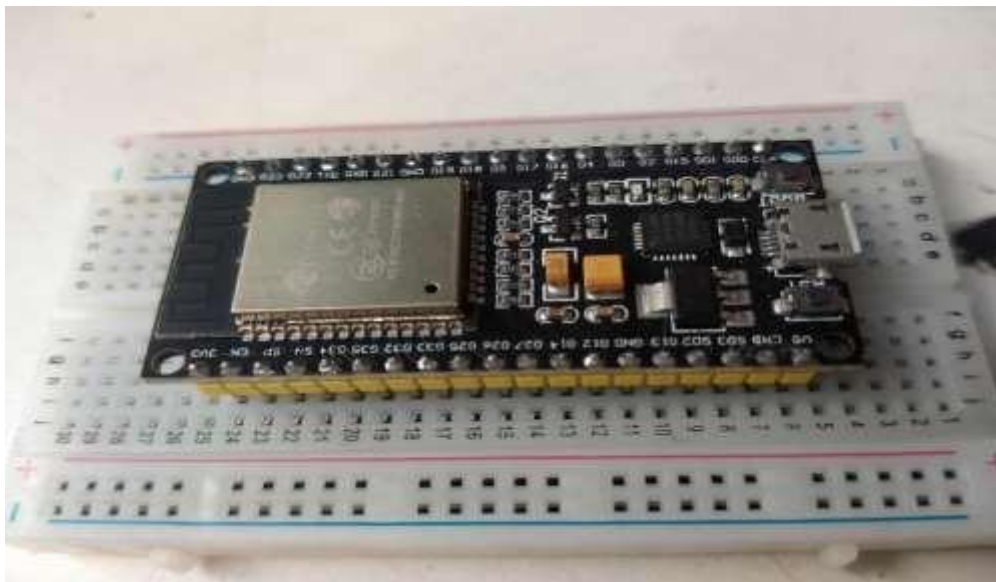
1. Arduino Uno (Unité Centrale Locale)

- **Rôle** : Cerveau principal. Il lit le capteur, pilote le relais/ventilateur, gère l'écran LCD et le Buzzer. Il formate les données pour l'envoi.
- **Tension** : 5V



2. ESP32 (Passerelle Wi-Fi)

- **Rôle** : Interface réseau. Il ne prend aucune décision. Il reçoit les données sur son port Série (Rx) et les pousse vers Firebase.
- **Tension** : 3.3V.



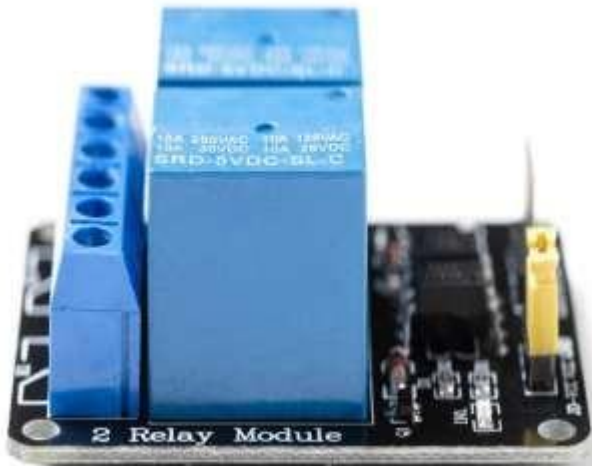
3. Capteur de Température (DHT11)

- **Connexion** : Branché directement sur l'Arduino.



4. Relais 5V & Ventilateur DC

- **Connexion** : Le relais est piloté par l'Arduino (Pin 7) pour fermer le circuit 5V du ventilateur.



5. Écran LCD I2C & Buzzer

- **Connexion** : Branchés sur l'Arduino pour l'interface homme-machine locale.



3. Architecture et Câblage

L'architecture matérielle a été adaptée pour correspondre à la logique logicielle définie.

3.1 La Gestion des Périphériques (Côté Arduino)

L'Arduino fournit l'alimentation 5V nécessaire au Relais, au LCD et au Capteur.

3.2 La Communication Série (Le Pont UART)

La communication entre les deux cartes se fait via le protocole Série (Tx/Rx) à une vitesse de 9600 bauds.

- Arduino Tx (Pin 1) -> ESP32 Rx (GPIO 16)
- Arduino Rx (Pin 0) -> ESP32 Tx (GPIO 17)
- GND Commun : Indispensable pour la référence de tension.

3.3 Schéma de Connexion Global (Mise à jour selon le Code)

1. Zone Arduino :

- DHT11 : Broche Analogique A1
- Relais (Ventilateur) : Broche Digitale 8
- Buzzer : Broche Digitale 7
- LCD I2C : Broches SDA (A4) / SCL (A5) ○ Tx :
Vers ESP32 Rx.

2. Zone ESP32 :

- Rx : GPIO 16 (Réception des données Arduino)
- Tx : GPIO 17
- Wi-Fi : Connecté au réseau "inwi Home".

4. Logique et Implémentation Logicielle

Le fonctionnement du système est régi par une machine à états distribuée entre les deux microcontrôleurs.

4.1 Algorithme de Fonctionnement

A. Logique de l'Esclave (Arduino Uno) : L'Arduino exécute une boucle de surveillance toutes les 3 secondes :

1. **Acquisition :** Lecture des valeurs d'humidité et de température via le capteur DHT11.
2. **Décision de Refroidissement :**
 - Si Température $\geq 21^{\circ}\text{C}$: Activation du Relais (Le ventilateur démarre).
 - SINON : Désactivation du Relais.
3. **Décision d'Alerte Locale :**
 - Si Température $> 25^{\circ}\text{C}$ (Seuil critique) : Activation du Buzzer et affichage "URGENT" sur l'écran LCD.
4. **Transmission :** Envoi des données vers l'ESP32 via le port série sous le format CSV : Température, Humidité, État Ventilateur.

B. Logique du Maître (ESP32) : L'ESP32 est en écoute permanente :

1. **Réception :** Attente d'une ligne de données sur le port UART.
2. **Parsing :** Découpage de la chaîne reçue pour extraire les valeurs numériques.
3. **Cloud :** Envoi des données vers la base de données Firebase en temps réel.
4. **Alerte Distante (WhatsApp) :**
 - Si Température $> 25^{\circ}\text{C}$ ET Alerte non active : Envoi immédiat d'un message WhatsApp "Alerte CRITIQUE".
 - Si Température toujours $> 25^{\circ}\text{C}$ après 15 secondes : Envoi d'un message de rappel.

- Si Température redescend $\leq 25^{\circ}\text{C}$: Envoi d'un message de confirmation "Retour à la normale".

4.2 Code Source : Contrôle Physique (Arduino Uno - C++)

```
#include "DHT.h"
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

const int DHT_PIN = A1;
const int DHT_TYPE = DHT11;
const int RELAY_PIN = 8;
const int BUZZER_PIN = 7; // Buzzer sur broche 7
const float TEMP_SEUIL = 21.0;

// Valeur critique pour URGENT
const float TEMP_CRITIQUE = 25.0;

// LCD 16x2 I2C (adresse 0x27)
LiquidCrystal_I2C lcd(0x27, 16, 2);

DHT dht(DHT_PIN, DHT_TYPE);

void setup() {
    Serial.begin(9600);
    pinMode(RELAY_PIN, OUTPUT);
    pinMode(BUZZER_PIN, OUTPUT); // Initialisation buzzer
    digitalWrite(RELAY_PIN, HIGH);
    digitalWrite(BUZZER_PIN, LOW); // Buzzer éteint par défaut
    dht.begin();

    // LCD initialisation
    lcd.init();
    lcd.backlight();
}
```

```

void loop() {
    delay(3000);

    float temp = dht.readTemperature();
    float hum = dht.readHumidity();

    if (isnan(temp) || isnan(hum)) return;

    int fan = (temp >= TEMP_SEUIL) ? 1 : 0;
    digitalWrite(RELAY_PIN, (fan == 1) ? LOW : HIGH);

    // ----- Gestion buzzer -----
    if (temp > TEMP_CRITIQUE) {
        digitalWrite(BUZZER_PIN, HIGH);    // buzzer activé
    } else {
        digitalWrite(BUZZER_PIN, LOW);     // buzzer désactivé
    }

    // ----- Affichage LCD -----
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Temp: ");
    lcd.print(temp, 1);
    lcd.print("C");

    lcd.setCursor(0, 1);
    if (temp > TEMP_CRITIQUE) {
        lcd.print("URGENT !");
    } else {
        lcd.print("          "); // vide la ligne
    }

    // ----- Envoi Serial -----
    Serial.print(temp, 2);
    Serial.print(",");
    Serial.print(hum, 2);
    Serial.print(",");
    Serial.println(fan);
    delay(3000);
}

```


4.3 Code Source : Passerelle IoT (ESP32 - MicroPython)

```
import machine
import network
import urequests
import time

# Variables globales pour la gestion des alertes SMS
LAST_SMS_TIME = 0
SMS_INTERVAL = 15 # Intervalle minimum entre deux SMS d'alerte (en secondes)
ALERT_ACTIVE = False # Indique si l'alerte (température > 25°C) est actuellement active

def send_sms(message):
    try:
        phone = "+0000000000000" # ton numéro complet avec indicatif
        apikey = "VOTRE_CLE_API" # clé API reçue depuis CallMeBot

        # encoder les espaces et caractères spéciaux
        message_encoded = message.replace(" ", "%20")

        url = "https://api.callmebot.com/whatsapp.php?phone={}&text={}&apikey={}".format(
            phone,
            message_encoded,
            apikey
        )

        response = urequests.get(url)
        print("SMS envoyé :", response.text)
        response.close()

        # Mettre à jour le temps du dernier SMS envoyé (NÉCESSAIRE ici)
        global LAST_SMS_TIME
        LAST_SMS_TIME = time.time()

    except Exception as e:
        print("Erreur envoi SMS:", e)

# --- Connexion WiFi ---
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
print("Connexion au WiFi...")
wlan.connect(SSID, PASSWORD)
```



```

for i in range(10):
    if wlan.isconnected():
        break
    time.sleep(1)
if wlan.isconnected():
    print("Connecté au WiFi:", wlan.ifconfig())
else:
    print("Impossible de se connecter au WiFi")
    raise SystemExit
uart = machine.UART(1, tx=17, rx=16, baudrate=9600) # TX/RX selon ton câblage

FIREBASE_URL = 'https://led-statue-a48fc-default-rtdb.firebaseio.com/'

def send_to_firebase(path, value):
    if wlan.isconnected():
        try:
            urequests.put(FIREBASE_URL + path + '.json', json=value).close()
            print(f"Donnée envoyée: {path} -> {value}")
        except Exception as e:
            print("Erreur Firebase:", e)
    else:
        print("Pas de connexion WiFi, données non envoyées")
while True:
    if uart.any():
        line = uart.readline()
        if line:
            line = line.decode().strip()
            print("Reçu de Arduino:", line) # Affichage pour debug

            # Essayer de convertir en float / int si possible
            try:
                parts = line.split(',')
                if len(parts) == 3: # Format attendu : "temp,hum,fan"
                    temp = float(parts[0])
                    hum = float(parts[1])
                    fan = int(parts[2])

                    # Envoyer à Firebase
                    send_to_firebase('/Temperature', temp)
                    send_to_firebase('/Humidity', hum)
                    send_to_firebase('/Fan', fan)

```

```

try:
    parts = line.split(',')
    if len(parts) == 3: # Format attendu : "temp,hum,fan"
        temp = float(parts[0])
        hum = float(parts[1])
        fan = int(parts[2])

        # Envoyer à Firebase
        send_to_firebase('/Temperature', temp)
        send_to_firebase('/Humidity', hum)
        send_to_firebase('/Fan', fan)

        # LOGIQUE D'ALERTE WHATSAPP
        current_time = time.time()
        # Suppression des déclarations 'global' ici pour éviter la SyntaxError

        if temp > 25:
            # Si la température dépasse 25°C
            if not ALERT_ACTIVE:
                # Premier message d'alerte (déclenchement)
                send_sms("⚠ Alerte CRITIQUE ! La température atteint {}°C. Action immédiate requise.".format(temp))
                ALERT_ACTIVE = True
            elif current_time - LAST_SMS_TIME >= SMS_INTERVAL:
                # Message de rappel (avec temporisation de 15s)
                send_sms("⚠ Alerte RAPPEL ! La température est toujours à {}°C.".format(temp))

            elif temp <= 25 and ALERT_ACTIVE:
                # Si la température redescend en dessous ou égale à 25°C ET que l'alerte était active
                send_sms("✅ Température sous seuil critique.")
                ALERT_ACTIVE = False

    except Exception as e:
        print("Erreur conversion UART:", e)
    time.sleep(0.1)

```

5. Conclusion :

Cette réalisation démontre l'efficacité d'une architecture distribuée. L'Arduino assure la sécurité immédiate des installations (refroidissement dès 21°C, alarme à 25°C) sans dépendre d'Internet, tandis que l'ESP32 offre une couche de supervision moderne via Firebase et WhatsApp. Ce système hybride combine la robustesse industrielle et la connectivité IoT.

